Supplementary Materials for

The Naibbe cipher: a substitution cipher that encrypts Latin and Italian as Voynich Manuscript-like ciphertext

Michael A. Greshko[a]*

[a]Independent Researcher, Washington, D.C., USA

*michael@michaelgreshko.com

Cryptologia. DOI: 10.1080/01611194.2025.2566408

This PDF contains:

Supplementary Material 1: Additional properties of Naibbe ciphertext

Supplementary Material 2: Voynichesque

Supplementary Material 3: Fitting the frequency-rank word type distribution of Voynich B with a model of a substitution cipher acting on plaintext unigrams

Supplementary Material 4: Random walk analysis

Supplementary Material 1: Additional properties of Naibbe ciphertext

S1.1. Hapax legomena and rare glyphs within word types

Because of the Naibbe cipher's handling of plaintext bigrams, about half of all Naibbe cipher word types in ciphertexts approaching the length of Voynich B appear only once, consistent with the VMS (Zandbergen 2024). What's more, the Naibbe cipher includes the rare glyphs <*x*> and <*g*> as a proof of concept for how a substitution cipher could accommodate rare Voynichese glyphs as exhibited by the VMS.

S1.2. Text interspersed within or on either side of an illustration

Because Naibbe ciphertext tokens and word types represent plaintext unigrams and bigrams, the plaintext message is broken up into many individual chunks anyway, making it straightforward to continue a sequence of tokens after some mid-sequence interruption, such as an intrusive illustration. This kind of wrapping around, and even through, illustrations is a hallmark of VMS text. While the unigram or bigram nature of Naibbe tokens makes them less than informative as individual illustration labels, a limitation discussed in the main text's Section 4, it also means that the Naibbe cipher can replicate what seem to be unusual clusters of VMS "labels," such as the two lines of tokens cutting through the bathing illustration in folio f75v.

S1.3. Asymmetrical word pairings

Much like many natural languages, the VMS exhibits asymmetrical word pairings (aka skewed pairs), in which for words *A* and *B*, the word pair *AB* occurs more often than *BA* (Caruana, Layfield, and Abela 2022). Naibbe ciphertexts do the same thing, with some word pairs occurring more than an order of magnitude more often than their reverses.

These pairings are most often pairs of unigram word types that encrypt asymmetrical pairs of letters in Latin and Italian, the most common being U and M, as UM is a common Latin word ending and a more common bigram than MU (S1 Table 1).

S1.4. Smooth, orderly handwriting

With practice, the Naibbe cipher can provide for smooth, orderly, and seemingly error-free writing akin to what is observed in the VMS. For the most part, scribes are simply copying down prescribed strings of Voynichese glyphs. In experimental encipherment sessions—including the one that resulted in the main text's Figure 5, a ciphertext encrypting some of Julius Caesar's *De bello Gallico* (Holmes 2025)—I have found that as long as the scribe is accustomed to writing the Voynichese glyph set, logging the prescribed glyph string in short-term memory and then copying it down often results in orderly writing, especially after doing an initial scratch-paper encryption.

Scratch-paper encryption also provides a mechanism for the most complex ligatures seen in the VMS, most often in the portions known as Voynich A, as seen with the flamboyant combined treatment of the glyph <*t*> within <*ctho*> <*cthey*> on line 12 of folio f8r. The easiest way to create such a ligature is to do an initial encryption, spot two <*cth*> glyphs that happen to be in adjacent ciphertext tokens, and then perform the ligature while copying down the final text. It is both a complication and a flourish, consistent with the work of an experienced scribe.

S1.5. Clustering of similar words

Puzzlingly, the VMS can exhibit clusters of words with similar structures, including identical prefixes (Timm 2015; Timm 2016). Naibbe ciphertexts can do the same, thanks to the word types that encrypt plaintext unigrams (S1 Table 2). For example,

because many of the primary table's word types for plaintext unigrams begin with <*qok*>, if a series of plaintext unigrams happens to be generated during respacing, the single likeliest outcome of the default encryption procedure is that at least some of the unigrams will start with <*qok*>. These kinds of <*qok*> clusters can be seen in the experimental Naibbe ciphertext in Figure 5 of the main text.

With Latin and Italian plaintexts, we would generally expect to see this kind of clustering in areas of the plaintext with high concentrations of the letters L-T, as these letters tend to have fairly high occurrence rates and are all adjacent to each other in the alphabet. The construction of the Naibbe cipher's tables is meant to emulate the possibility—echoing the Polygraphia III cipher—that at least some VMS word types are generated by listing out identical prefixes and then modifying suffixes on a letter-by-letter basis (Trithemius 1564; Hermes 2022). To a truly eerie degree, the frequencies of common Voynich B word types lend themselves remarkably well to a Naibbe-style 6-table grouping, with 5 of the 6 Naibbe tables' unigram options consisting mostly of word types with one of 5 conserved beginnings—<*qok*>, <*ok*>, <*qot*>, <*ot*>, and <*olk*>—and a highly regularized scheme of varying suffixes (S1 Table 2).

The Naibbe tables' unigram word types also allow for what Timm and Schinner (2020) called context-dependent self-similarity: the VMS property that pairs of common words on any given page tend to resemble each other. If a Naibbe scribe used one cipher table especially often while encoding the unigrams on a given page—either as the primary table within the cipher's normal card-drawing procedure or as a non-random preference—that table's structurally similar word types would become a sizable fraction of the page's overall word types.

S1.6. Repetition of identical words

To the extent that Naibbe ciphertext word types repeat (0.18±0.04% of the time, lower than observed VMS rates), this repetition can occur in at least three different ways:

1. Letter duplication within a plaintext word (e.g., *radii*, *tutti*, *anno*).
2. The end of one plaintext word and the beginning of the next plaintext word happen to be the same letter.
3. The encoding of Roman numerals, especially for C, I, M, and X.

In all three cases, the likeliest way to get a string of repeating words is for those words to encode plaintext unigrams with a conserved table, often the primary or a secondary table.

For example, based purely on a comparison between their proportional frequencies within Voynich B and the word type frequencies needed within the Naibbe tables, I constructed the Naibbe tables so that the word types <*shedy*> and <*qokedy*> both stand for a unigram I (in α and β_1, respectively). The additive form of the Roman numeral 4, IIII, was still in use in medieval Europe (Burnett 2020), and in principle, one could encode a plaintext IIII using the Naibbe cipher as something like <*shedy*> <*qokedy*> <*shedy*> <*qokedy*> or even as <*qokedy*> <*qokedy*> <*qokedy*> <*qokedy*>. While this work does not purport to be a decryption of the VMS—or even confirmation that the VMS is a ciphertext—it is intriguing that this method can generate the phrase <*shedy*> <*qokedy*> <*shedy*> seen in folios 83 and 84, as well as one of the VMS's most dramatic instances of word repetition: the four <*qokedy*> in a row in line 13 of folio f75r.

In addition, word repetition from one line to the next may occur through the non-random use of previous lines of ciphertext as a basis for habitually encrypting the next line of plaintext. For example, if a scribe used the word <*chedy*> to encrypt a plaintext unigram E in one line and was then faced with another plaintext unigram E in the next line, the scribe may simply recycle the <*chedy*> assignment without going through the formal card-drawing procedure, similar to the "self-citation" process that Timm and Schinner (2020) invokes.

In principle, it should be possible to increase a Naibbe ciphertext's rate of word repetition to VMS-like levels (0.8–0.9%) by encrypting a Latin or Italian plaintext that contains a large proportion of Roman numerals, such as a timeline or diary with a series of entries that all include various years. In this case, one would expect higher repetition rates for ciphertext word types in the primary table that encrypt the unigram versions of C, I, M, and X, the letters most likely to be repeated multiple times within a single Roman numeral.

S1.7. Word similarity networks

Timm and Schinner (2020) reported that unlike natural languages, the word types within the VMS exhibit a deep connection between token frequency and the number of similar tokens. When analyzed as a network, where individual word types represent nodes that are connected by an edge if the two words differ by just one glyph (i.e., a Levenshtein distance of 1, after Levenshtein (1966)), Timm and Schinner (2020) also reported that VMS word types group into four broad domains: words similar to <*daiin*>; words similar to <*ol*>; words similar to <*chedy*>; and a catch-all domain for everything else. The word types within Naibbe ciphertexts exhibit similarity networks whose connectivity and domains are strikingly reminiscent of the VMS. The word types within

a Naibbe ciphertext encrypting 32,000 letters of a Latin "alchemical herbal" (Ponzi 2023) exhibit a network structure (S1 Figure 1) that resembles the one reported by Timm and Schinner (2020) for the entire VMS, with four broad domains of word types and a longest network path of 21, identical to the VMS. In addition, the Naibbe word network's broad structure and connectivity resemble those of Voynich B, as reported in the supplementary materials of Timm and Schinner (2020).

S1.8. Topic-dependent variation in word type frequencies

Through the lens of the Naibbe cipher, the VMS topic modeling results of Reddy and Knight (2011) and Sterneck, Polish, and Bowern (2021) may indicate the detection of ciphertext word types that encode plaintext unigrams and bigrams whose frequencies vary by plaintext topic. For example, Latin herbals often include the word *herbe* so may include a high frequency of the bigram HE or the unigram B relative to texts on other subjects, while astronomical treatises on the moon (*lunae* in Latin) may include a relatively high frequency of the bigram LU.

I provide an initial demonstration of this effect with the Naibbe cipher by encrypting a 32,000-letter Latin plaintext where each half is about a single known topic and then analyzing the word type frequencies across each half of the ciphertext. The first 16,000 letters of the plaintext come from the Latin "alchemical herbal" (Ponzi 2023), while the second half is the first 16,000 letters of Grosseteste's astronomical treatise *De sphaera* (King 2008). I find that ciphertext word types' frequencies can vary considerably between the two sections, with word types shared between the two sections varying in their per-section frequencies by an order of magnitude or more and some word types appearing only in one section or the other (S1 Table 3). The most "relevant" words for a given section, as weighed by their overall and section-specific

frequencies, correspond to frequently used, topic-specific plaintext nouns and verbs. Within the herbal section, for instance, many of the most relevant word types encode E, B, V, H, and ER: seemingly picking up on the herbal's frequent use of both the words herbe and herba and words such as *vino*, *vivi*, and *pulveriza* that happen to all contain the letter V.

I confirm and expand on these results by performing two full topic modeling analyses of both Voynich B and a ~21,000 Naibbe ciphertext created by encrypting this section's standard 32,000-letter plaintext, a four-part plaintext made of equal parts Divina Commedia, Natural History, De sphaera, and the 14th-century alchemical herbal. Taking after Sterneck, Polish, and Bowern (2021), I perform a latent Dirichlet allocation (LDA) analysis, using Enderle (2017)'s GUI for MALLET (McCallum 2002). Using a custom Python script, I also perform *k*-means clustering on a principal components analysis (PCA) of both Voynich B and the Naibbe ciphertext, after having numerically represented each word in each document using term frequency-inverse document frequency (TF-IDF) vectorization within Microsoft Excel. The optimal number of *k*-means clusters is determined by silhouette score (Rousseeuw 1987), with a minimum cluster number of 3 and a maximum of 10, for ease of broad interpretability.

Consistent with the findings of Sterneck, Polish, and Bowern (2021), LDA largely fails to recover topics in either Voynich B or the Naibbe ciphertext because LDA tends to create a topic consisting of globally common word types—such as <*chedy*>, <*shedy*>, and <*ol*>—and then assigns most subsets of a given document to it. However, also consistent with Sterneck, Polish, and Bowern (2021), topic modeling based on TF-IDF detects meaningful clusters of word types within Voynich B that are associated with well-characterized sections of the VMS (S1 Figure 2a), with a finer

structure and a greater number of clusters preferred when Voynich B word types are parsed by both section and the hands of Davis (2020)'s five scribes (S1 Figure 2b).

Intriguingly, when the Naibbe ciphertext is broken into fourths—as if each underlying plaintext section were a visually distinct section within a synthetic VMS—and then subjected to the same analysis, Naibbe word types show meaningful clustering based on the identity of the underlying plaintext section (S1 Figure 2c). The most informative words within a given section per TF-IDF are either plaintext bigrams that are diagnostic of the differences between Latin and Italian or plaintext bigrams associated with section-abundant, topic-specific nouns and verbs.

For example, TF-IDF suggests that the 4 most informative words within the Divina Commedia section of the Naibbe ciphertext are <*lcheedal*>, <*lcher*>, <*lcheeoy*>, and <*lcheaiin*>, all of which are rare ways of encrypting CH, which is abundant in Italian (*Divina Commedia*) and rare in Latin (the rest of the plaintext). Similarly, the 5 most informative words per TF-IDF within the *De sphaera* section—<*opeeo*>, <*oteed*>, <*rees*>, <*ydy*>, and <*teeey*>—translate to US, US, OT, CI, and RC, respectively, picking up on the section's uniquely frequent use of topic-specific plaintext words such as *circulus* (circle) and *motu*/*motus* (movement).

These results conclusively show that Naibbe ciphertexts can exhibit topic-dependent variation in ciphertext word frequencies, in a manner consistent with the findings reported for the VMS by groups including Reddy and Knight (2011) and Sterneck, Polish, and Bowern (2021).

S1.9. Morphological complexity

Lindemann (2022) suggested two metrics for morphological complexity by which the VMS could be compared to natural languages: the moving-average type-token ratio

(MATTR), and the most common words measure (MCW). The MATTR of a text measures the average number of unique word types within a rolling window of a token length that Lindemann (2022) fixes as 1,000 tokens. The MCW_N of a text is the fraction of tokens occupied by the N commonest word types; Lindemann (2022) finds that languages show the most variance when $N = 70$.

Naibbe ciphertexts reliably come close to replicating the VMS's observed MATTR and MCW_{70} values. Naibbe ciphertexts reliably achieve an MCW_{70} value of 0.44±0.01, consistent with the VMS's reported range in Lindemann (2022) of ~0.40-0.44. With a MATTR of 0.56±0.01, Naibbe ciphertexts overshoot what Lindemann (2022) reported for the VMS (0.49-0.53). However, Naibbe ciphertexts sit closer to the VMS than the *Tanach Bereshit*, *Rettorica*, *Codex Suprasliensis*, and *Mabinogion*, Lindemann (2022)'s reference manuscripts (S1 Figure 3).

S1.10. Degree of token autocorrelation versus sequence offset

While evaluating Hermes (2022)'s suggestion that an artificial-language cipher like Polygraphia III could explain the properties of the VMS, Timm and Schinner (2024) define and elaborate on a problem known as the token autocorrelation problem. Let t_i denote the sequence of tokens in a manuscript, such as the VMS or a Naibbe ciphertext, that begins with the ith token. We also define the sequence offset k, such that t_{i+k} begins with the $(i+k)$th token but is otherwise identical. We define the autocorrelation $C_{i,k}$ of t_i with sequence offset k as:

$$C_{i,k} = \frac{A_{i,k} - D_{i,k}}{A_{i,k} + D_{i,k}} \quad (1)$$

Where $A_{i,k}$ and $D_{i,k}$ denote the number of agreeing and the number of non-agreeing sequence members, respectively, when comparing t_i with t_{i+k} on a token-by-token basis from the start of each sequence.

Timm and Schinner (2024) note that in substitution ciphers where the text is broken up entirely into individual letters and each letter can be represented by one of N unique words, each within one of N alphabets that are predictably cycled through, a token sequence will strongly correlate with itself when offset by multiples of N. This autocorrelation leads to periodic peaks in C values that are not observed in the VMS.

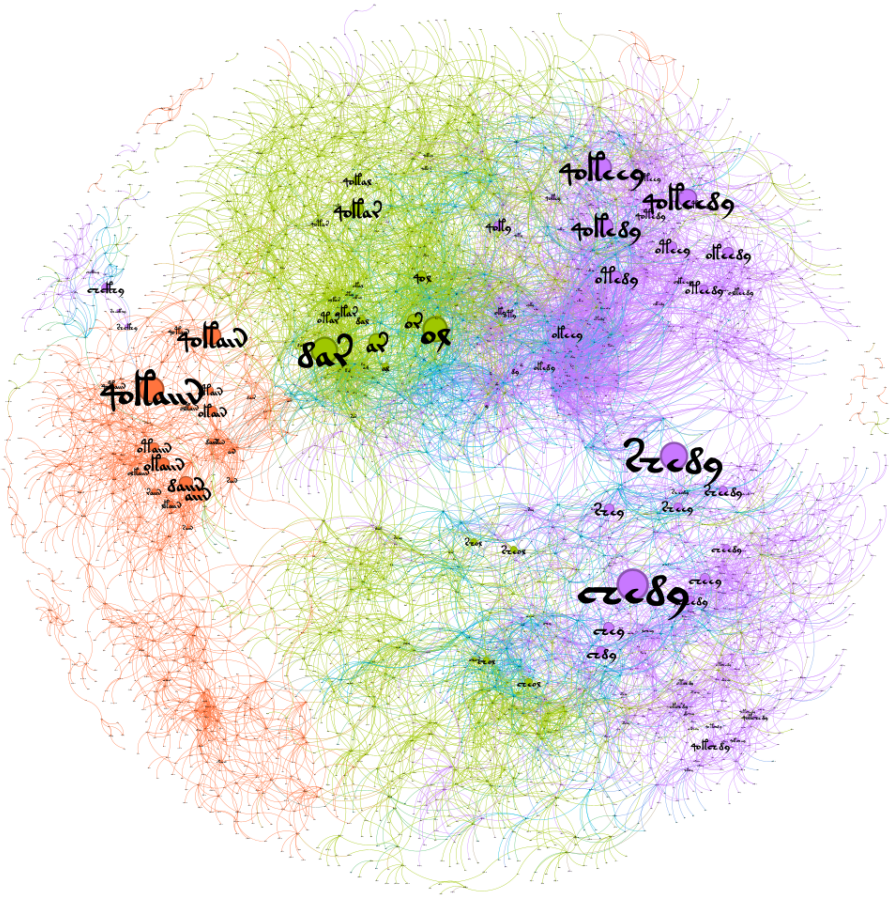
Unlike the above example, which describes the general construction of Polygraphia III, the Naibbe cipher encrypts a unigram-bigram plaintext and randomly selects substitution options on a letter-by-letter basis, which is enough to meaningfully change the situation. A Naibbe ciphertext encrypting the first ~31,000 letters of Dante's *Divina Commedia* replicates Timm and Schinner (2024)'s reported absence of periodic autocorrelation peaks within the "Stars" section of the VMS, folios f103r through f116r (S1 Figure 4).

Word pairing	Plaintext	Count of pairing	Count of reversed pairing	Relevance
<qokain> <qokar>	UM	14	1	12.13
* <chey>*	AC	10	0	10.00
<okain> <qokar>	UM	11	1	9.17
<qokchedy> <dar>	HE	9	0	9.00
<oteedy> <qokchdy>	RB	8	0	8.00
* <qokeey>*	AS	14	4	7.78
<chedy> <qokaiin>	ET	15	5	7.50
<chedy> <qokeedy>	ER	17	7	7.08
<qokchdy> <chedy>	BE	7	0	7.00
<qokain> <okar>	UM	7	0	7.00

S1 Table 1. The top 10 most relevant asymmetrical word pairings within a representative ~20,000-token, 78-card Naibbe ciphertext encrypting the Latin "alchemical herbal." Relevance is calculated as $R = \frac{A(A-B)}{A+B}$, where A is the count of a given word pairing and B is the count of the reversed word pairing.

Letter	α	β_1	β_2	β_3	γ_1	γ_2
A	* *	*<or>*	*<qol>*	*<chdy>*	*<chol>*	*<cheol>*
B	*<qokchdy>*	*<okchdy>*	*<otchdy>*	*<sheckhy>*	*<qotchdy>*	*<olkchdy>*
C	*<chey>*	*<okedy>*	*<lkaiin>*	*<shckhy>*	*<sar>*	*<lchey>*
D	*<cheey>*	*<cheedy>*	*<lkain>*	*<chor>*	*<chckhey>*	*<olkedy>*
E	*<chedy>*	*<ar>*	*<dar>*	*<dar>*	*<chckhy>*	*<lchedy>*
F	*<qokchey>*	*<okchey>*	*<otchey>*	*<shal>*	*<qotchey>*	*<olkchey>*
G	*<qokchy>*	*<okchy>*	*<otchy>*	*<shar>*	*<qotchy>*	*<olkchy>*
H	*<qokchedy>*	*<okchedy>*	*<otchedy>*	*<shy>*	*<qotchedy>*	*<olkchedy>*
I	*<shedy>*	*<qokedy>*	*<shey>*	*<otedy>*	*<raiin>*	*<qotedy>*
L	*<qokal>*	*<okal>*	*<otal>*	*<saiin>*	*<qotal>*	*<olkal>*
M	*<qokar>*	*<okar>*	*<otar>*	*<sain>*	*<qotar>*	*<olkar>*
N	*<daiin>*	*<dal>*	*<dain>*	*<dy>*	*<dair>*	*<dam>*
O	*<aiin>*	*<al>*	*<ain>*	*<y>*	*<air>*	*<am>*
P	*<qoky>*	*<oky>*	*<oty>*	*<sheody>*	*<qoty>*	*<olky>*
Q	*<qokol>*	*<okol>*	*<otol>*	*<shody>*	*<qotol>*	*<olkol>*
R	*<qokeedy>*	*<okeedy>*	*<oteedy>*	*<sheol>*	*<qoteedy>*	*<olkeedy>*
S	*<qokeey>*	*<okeey>*	*<oteey>*	*<shol>*	*<qoteey>*	*<olkeey>*
T	*<qokaiin>*	*<okaiin>*	*<otaiin>*	*<sheey>*	*<qotaiin>*	*<olkaiin>*
U	*<qokain>*	*<okain>*	*<otain>*	*<sheedy>*	*<qotain>*	*<olkain>*
V	*<qokey>*	*<okey>*	*<otey>*	*<shor>*	*<qotey>*	*<olkey>*
X	*<qokor>*	*<okor>*	*<otor>*	*<shaiin>*	*<qotor>*	*<olkor>*
Y	*<qokdy>*	*<okdy>*	*<otdy>*	*<shain>*	*<qotdy>*	*<olkdy>*
Z	*<qokeeedy>*	*<okeeedy>*	*<oteeedy>*	*<sheeey>*	*<qoteeedy>*	*<olkeeedy>*

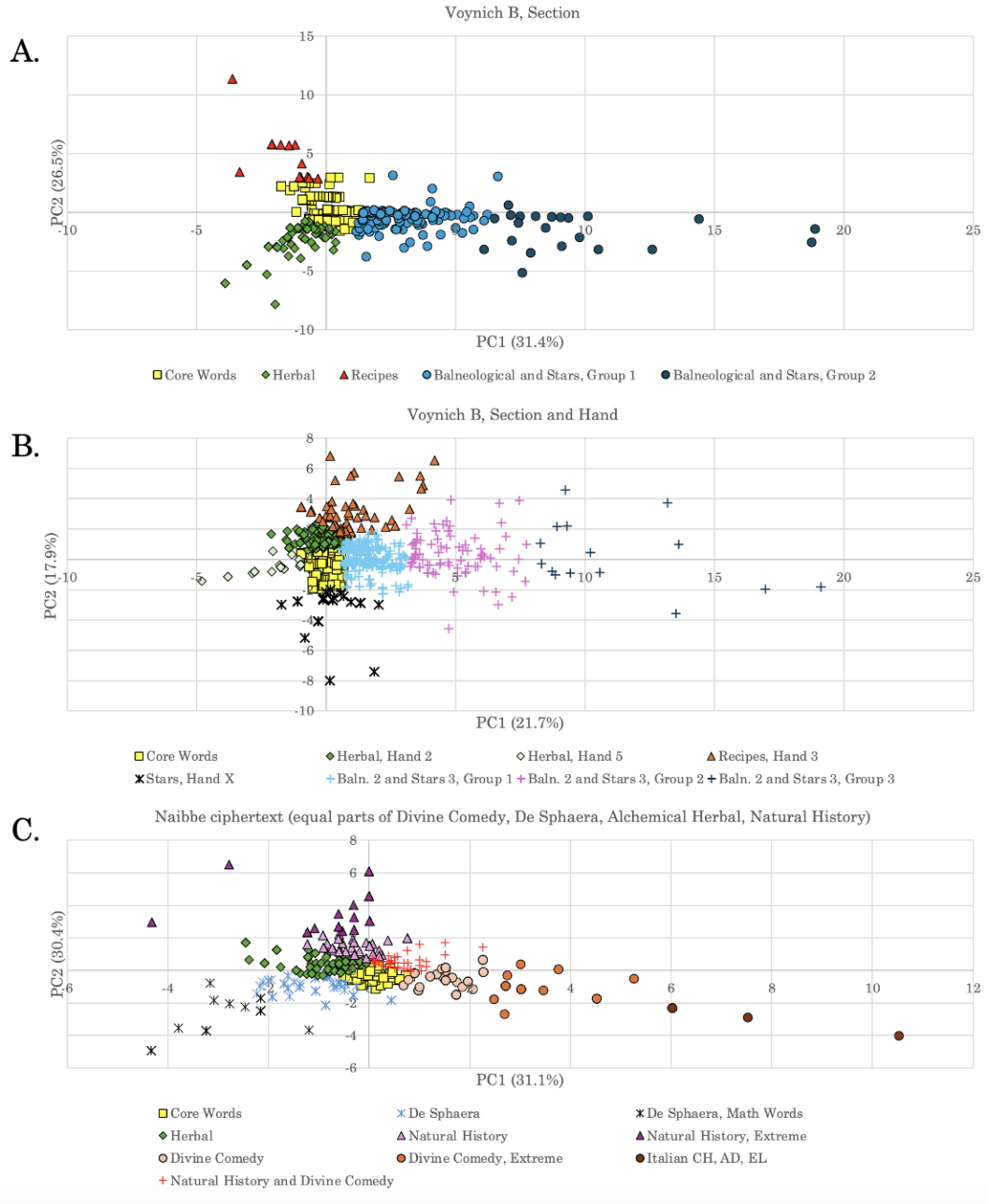
S1 Table 2. Naibbe ciphertexts can replicate the frequencies of many word types within Voynich B even while the tables' unigrams word types are rigorously organized by prefix and suffix. The tables were designed to reduce the edit distance between both the unigram word types for a table's adjacent alphabet letters and the multiple unigram word types for a given letter, consistent with Trithemius's method of creating words for his artificial-language cipher Polygraphia III (Trithemius 1564)



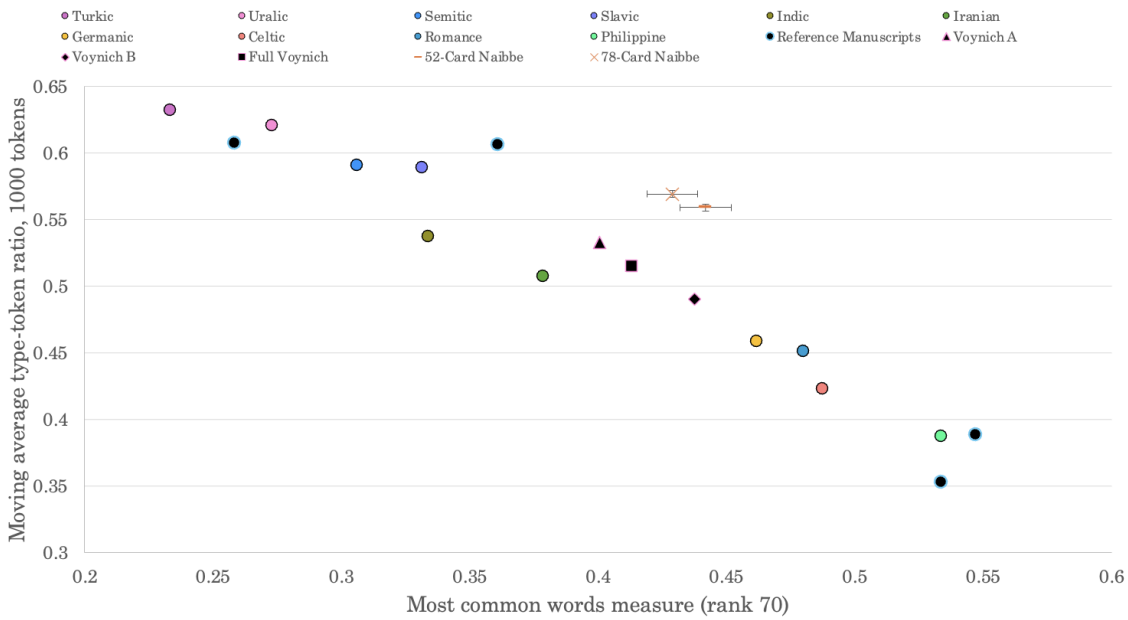
S1 Figure 1. Network of the 3,679 word types similar to the 1,200 commonest word types within a 78-card Naibbe ciphertext encrypting 32,000 letters of the Latin "alchemical herbal" (Ponzi 2023). Two words are called "similar" if they differ by only 1 glyph (i.e., have an edit distance of 1). A node's size corresponds to the word's frequency within the Naibbe ciphertext. To highlight words similar to <*daiin*>, <*ol*>, and <*chedy*>, I use the same color-coding scheme as Timm and Schinner (2020). All nodes for words that contain <*i*> are orange. Nodes of words containing <*ol*>, <*or*>, <*ar*>, <*al*>, or ending with <*am*> or <*os*> are green. Nodes of words ending in <*d*> or <*y*> are purple. All other nodes are blue.

Word type	Plaintext	Count in alchemical herbal	Count in *De sphaera*	Ratio	Relevance
<*dar*>	E	209	164	1.27	25.21
<*ol*>	A	166	127	1.31	22.10
<*qokchdy*>	B	53	22	2.41	21.91
<*sheckhy*>	B	27	3	9.00	21.60
<*qokey*>	V	20	1	20.00	18.10
<*cheedy*>	ER	31	12	2.58	13.70
<*qokchey*>	F	25	8	3.13	12.88
<*chedy*>	E	235	211	1.11	12.65
<*okchdy*>	B	29	12	2.42	12.02
<*shor*>	V	12	0	10.00	12.00

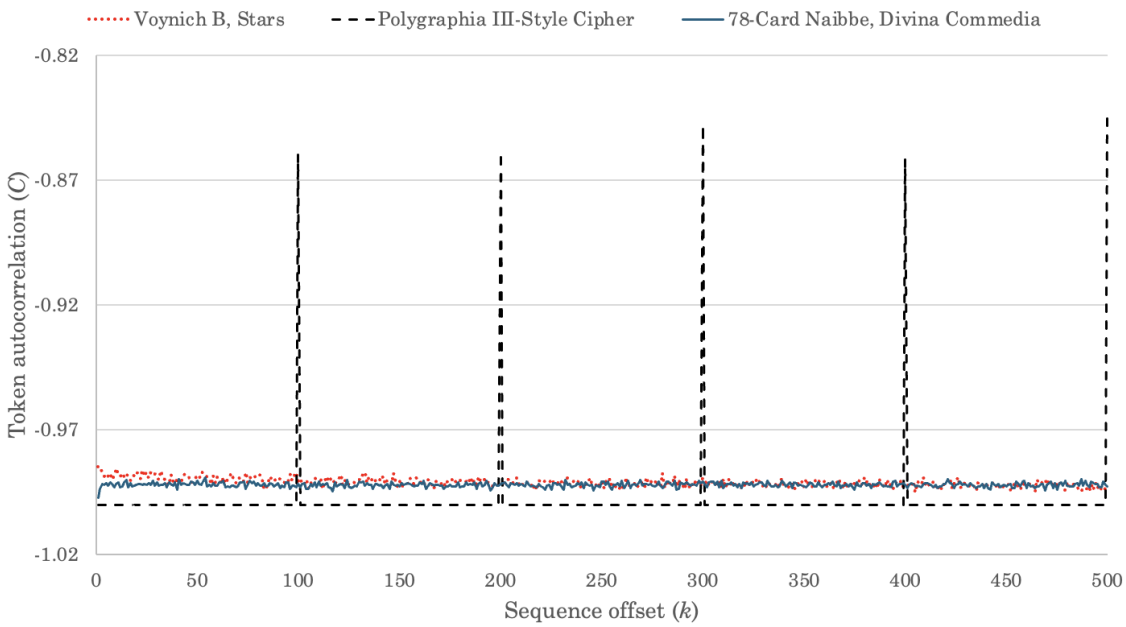
S1 Table 3. The top 10 most relevant words within a 78-card Naibbe ciphertext encrypting 16,000 letters of the Latin "alchemical herbal" (Ponzi 2023), relative to a Naibbe ciphertext encrypting 16,000 letters of Grosseteste's *De sphaera* (King 2008). Relevance is calculated as $R = \frac{A(A-B)}{A+B}$, where A is the count of a given word type in the herbal section and B is the count of the same word type in the *De sphaera* section. These word types correspond to topic-specific words in the herbal's plaintext, such as *herbe*, *vino*, *vivi*, and *pulveriza*. Note that <*cheedy*> is a rare instance of decryption ambiguity; the word type is supposed to be reserved for a unigram D.



S1 Figure 2. *k*-means clustering of a principal components analysis (PCA) done on a TF-IDF vectorization of the word types within: (a) Voynich B, where document sections are taken to be the VMS's well-characterized sections; (b) Voynich B, where document sections are broken down by VMS section and scribal hand, after Davis (2020); and (c) a four-plaintext Naibbe ciphertext. TF-IDF analysis universally recovers groupings associated with the known (Naibbe ciphertext) or inferred (Voynich B) topic of the text. Cluster labels are interpretive.



S1 Figure 3. The moving-average type-token ratio (MATTR) and most common words (MCW) measure for a variety of language families, as calculated with a MATTR window of 1,000 tokens and MCW for the 70 commonest words, after Lindemann (2022). Naibbe ciphertexts consistently produce MCW_{70} and MATTR values that are closer to the VMS than Lindemann (2022)'s reference manuscripts (black circles with aqua borders), which left to right are the *Tanach Bereshit*, *Codex Suprasliensis*, *Rettorica*, and *Mabinogion*. Error bars are $\pm 1\sigma$.



S1 Figure 4. Token autocorrelation versus sequence offset of the VMS "Stars" section (f103r-f116r; dotted red line) and Dante's *Divina Commedia* encrypted by a cipher similar to Trithemius's Polygraphia III (Trithemius 1564; Hermes 2022) that cyclically maps plaintext letters to ciphertext word types via a 100-column table (dashed black line). An 8,000-token, 78-card Naibbe ciphertext encrypting Dante's *Divina Commedia* (solid blue line) replicates the observed token autocorrelation of the VMS.

Supplementary Material 2: Voynichesque

Within Voynichesque, VMS-present combinations of slots 0-3 of Zattera (2022)'s slot alphabet generate the full list of options for the first cipher alphabet (Alphabet 1). Similarly, slots 4-7 generate the full options for the second alphabet (Alphabet 2), and slots 8-10 generate the full options for the third alphabet (Alphabet 3). During each run of Voynichesque, each of the three full lists of alphabet options is randomly subsampled to generate a 26-letter alphabet, subject to tuning parameters that govern how many letters in the alphabet are encrypted with strings of length 1, 2, 3, or 4. The terminal <*y*> glyph in Zattera (2022)'s slot 11 is treated as a null, applied to the ends of ciphertext words at a globally set probability. To mimic the common VMS word type <*daiin*>, we also hold that if an Alphabet 3 glyph string encrypts a unigram letter, a null <*d*> is appended to the beginning of the glyph string. We define various rules that govern: how alphabets are generated; which alphabets can be used to encode unigrams and bigrams; and the rules that govern those alphabets' applications. Trigrams are encrypted with all three cipher alphabets, applied left-to-right in 1-2-3 order.

At first glance, Voynichesque ciphertext resembles the repetitive quality of the VMS and replicates some of the VMS phenomenology, such as the proximity of similar-looking word types (S2 Figure 2; see <*daiiny*> <*daiiiny*> <*daiiin*> in the second line of ciphertext). While Voynichesque does not mimic the VMS with high fidelity, it does provides a framework in which we can map plaintext letters onto VMS-present, grammar-valid strings of Voynichese glyphs while also reproducing the VMS's token and word type length distributions. We can then tune Voynichesque and generate random variants of the cipher.

S2.1. Generation of cipher alphabets

For each run of Voynichesque, we randomly subsample each cipher alphabet's full list of options based on the options' glyph lengths as reported by Zattera (2022), using four natural-number parameters that must add up to 20 (i.e., discretizing percentages to the nearest 5%, to mimic an analog probability distribution). Each parameter determines the percentage of cipher alphabet options that must be a specific glyph length according to Zattera (2022). We typically set these four parameters to 3:9:7:1, based on the average observed distribution of VMS-present glyph strings among all combinations of Zattera slots 0–3, 4–7, and 8–10.

During each run of Voynichesque, each cipher alphabet's options of the same glyph length are assigned a randomly generated number between 0 and 1 and are then ranked. We take whatever number of top-ranked glyph strings are required to fulfill the cipher alphabet's total number of required strings of that length.

For example, if the four alphabet parameters are set to 3:9:7:1, each cipher alphabet must have 15% of their options be of glyph length 1, 45% of glyph length 2, 35% of glyph length 3, and 5% of glyph length 4. Because Voynichesque was originally designed for English, there are 26 alphabet letters, so our randomly generated cipher alphabet must have $0.15 \times 26 \sim 4$ options with glyph length 1. We then randomly rank all the length-1 options within each cipher alphabet's full list of options and use only the top 4. We perform the same operation for length-2, length-3, and length-4 options. Once we set a randomly generated 26-string cipher alphabet, we then randomly assign each string within the alphabet to a plaintext letter by generating a random number between 0 and 1 for each of the 26 strings, ranking these numbers, and then making plaintext letter assignments based on rank (where A = 1, B = 2, etc.).

Because we are subsampling each cipher alphabet's full list of options, we are restricting the total number of word types we can generate during a given run to a fraction of what is observed in the VMS, with some knock-on effects on calculated first-order entropy and conditional character entropy. However, this setup also allows us to efficiently generate large-number statistics, which at a minimum provides confidence in the directional signals gleaned from Voynichesque.

S2.2. Parametrization of Voynichesque

To probe the relationship between the nature of a plaintext and the VMS's statistical properties in the context of a VMS-mimic substitution cipher, I have parameterized the implementation of Voynichesque. Every major decision point in the encipherment procedure—including the generation of cipher alphabets, the alphabets' mappings to plaintext letters, and the distribution of plaintext n-gram lengths—can be governed by comparing the value of a manually set parameter with a random number on a token-by-token basis.

S2.2.1. Probability of generating "Voynichesque A" and "Voynichesque B" word types

We define a "Voynichesque A" word type as a plaintext bigram that is encrypted by applying Alphabet 2 to the bigram's first letter and Alphabet 3 to the bigram's second letter. By contrast, a "Voynichesque B" word type is generated by applying Alphabet 1 to the bigram's first letter and Alphabet 3 to the bigram's second letter. This setup is meant to qualitatively mimic the presence of and general structure of word types within Voynich A and Voynich B (aka Currier A and Currier B).

We can control the proportions of these bigram word types through use of a globally set parameter between 0 and 1. For every token of ciphertext, we generate a

random number between 0 and 1 and compare it with the globally set parameter. If the random number is greater than the parameter and the token encrypts a bigram, we use the Voynichesque A word type. If the random number is equal to or less than the parameter for a bigram, we use the Voynichesque B word type.

S2.2.2. Probability of using a given alphabet to encrypt a plaintext unigram

Within Voynichesque, either Alphabet 2 or Alphabet 3 can be used to encrypt a plaintext unigram. We set a global parameter between 0 and 1 and then compare that parameter to a randomly generated number between 0 and 1 on a token-by-token basis. If the random number is less than or equal to our parameter, we choose Alphabet 2; if not, we choose Alphabet 3.

S2.2.3. Probability of applying <y> *as a null suffix*

We set a global parameter between 0 and 1 that determines the probability of appending a null suffix <*y*> to a ciphertext token. On a token-by-token basis, we generate a random number between 0 and 1 and compare its value to the <*y*> parameter. If the random number is less than our parameter, we append a <*y*> null suffix. If not, we do not append a <*y*>.

S2.2.4. Distribution of plaintext n-*gram lengths*

Within Voynichesque, plaintexts are stripped of their original spacing and then respaced into a mix of unigrams, bigrams, and trigrams. Two globally set parameters, each between 0 and 1, control the probabilities that a given insertion of a space creates a unigram, bigram, or trigram. The parameter *u* controls the probability of unigram

creation; the parameter t controls the probability of trigram creation. The probability of bigram creation b is defined as:

$$b = 1 - (t + u) \quad (1)$$

We also require that $u + b + t = 1$ and that u, b, and t all be non-negative. To respace the plaintext, a random number r_n is generated between 0 and 1 for the nth token and then compared against the following quantities:

- If $r_n < u$, insert a space into the remaining unspaced plaintext 1 letter after the end of the previous token.
- If $u \leq r_n < u + b$, insert a space into the remaining unspaced plaintext 2 letters after the end of the previous token.
- If $r_n \geq u + b$, insert a space into the remaining unspaced plaintext 3 letters after the end of the previous token.

S2.3. Logging Voynichesque outputs

Within the framework outlined above, I can set tuning parameters, generate variants of Voynichesque that abide by those parameters, and then use these variants to encrypt a plaintext of my choosing: in this case, the first several thousand words of Book 16 of Pliny's *Natural History* (Thayer 2025). This text was chosen as the reference for Voynichesque because of its Latin language, length, and VMS-like subject matter (various plants and their uses). For each run of Voynichesque, we generate a single-use ciphertext that cuts off after 10,000 tokens. For each ciphertext, Voynichesque then outputs and logs the following:

- Character entropy for all 10,000 tokens, where <*ch*>, <*sh*>, <*cfh*>, <*ckh*>, <*cph*>, and <*cth*> are treated as glyphs as per Stolfi (2000), using a modified version of EVA.
- Conditional character entropy for the first 5,000 tokens, using modified EVA.
- The percentage of tokens with glyph lengths of a given length, recorded for glyph lengths 1–12 as defined by Stolfi (2000).
- The percentage of unique word types with glyph lengths of a given length, recorded for glyph lengths 1–12 as defined by Stolfi (2000).
- The average of 5 measurements of the ciphertext type-token ratio, calculated for a non-overlapping 1,000-token tranche within the first 5,000 tokens.
- The user-set parameters used to govern the run.
- A code that summarizes the run's three random cipher alphabets.

S2.4. Decrypting Voynichesque ciphertexts

When reading a Voynichesque ciphertext word left to right, the longest continuous sequence of Voynichese glyphs that can be interpreted as the substitution for a single plaintext letter when referencing our cipher alphabets is interpreted as such. To include this functionality, we define Alphabet 1 to end with Zattera's slot 3, which features the standalone glyphs <*f*>, <*k*>, <*p*>, and <*t*>. Seeing one of these glyphs means that the glyph and everything to the left of it maps onto a single letter within Alphabet 1. Similarly, Alphabet 2 is the only alphabet where glyph strings can begin with <*ch*>, <*sh*>, <*cfh*>, <*ckh*>, <*cph*>, <*cth*>, or <*e*>.

In the intentionally rare event that the cipher alphabets permit ambiguities in the readings of a given token (e.g., if <*qoiid*> is meant to be parsed as <*q*> <*oiid*> or as <*qo*> <*iid*>), the decoder writes down all possible readings of the token based on the cipher alphabets and then uses context clues after the fact to select the correct reading.

If any of the cipher alphabets happen to overlap in their glyph strings—which we generally try to avoid—then we accept that we have inadvertently made our cipher polyphonic, in which one ciphertext glyph string maps to 2 or more plaintext letters. In this case, the decoder also writes down all possible readings of the questionable token based on the cipher alphabets and uses context clues to select the correct reading.

S2.5. Population-level behavior of randomly generated Voynichesque variants

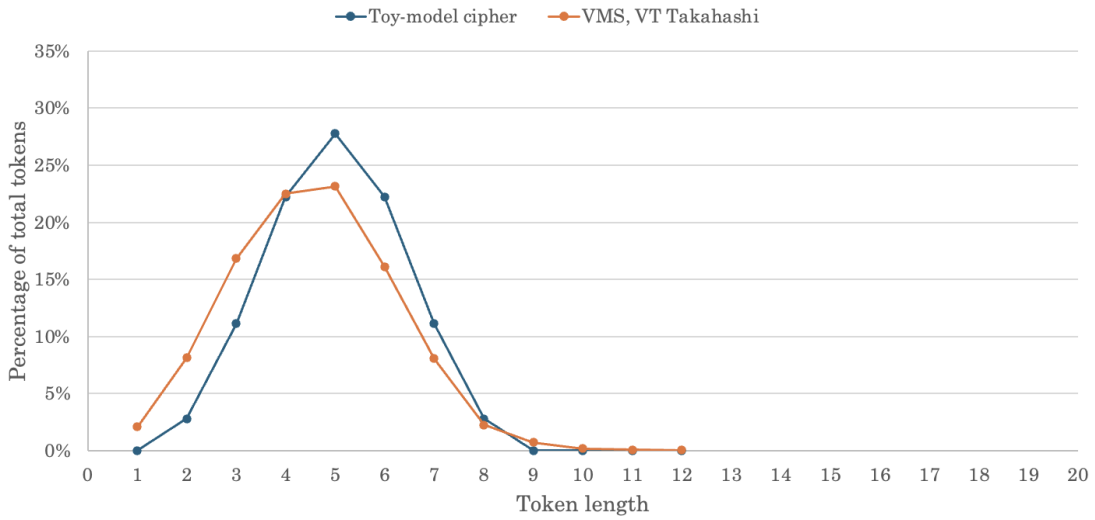
To elaborate on the argument set forth in Section 3.3 of the main text, there is no getting around the general problem of trigram-rich plaintexts yielding ciphertexts whose word type length distributions and entropy values do not reliably replicate the VMS at the same time—so much so that I posit that the VMS's word type length distribution places a powerful constraint on any substitution cipher proposal for the VMS. If, for instance, Voynichesque were altered so that it uses 4 3-glyph-long alphabets versus the 3 4-glyph alphabets presented above, tensions would simply kick in with 4-grams and get worse from there, while making it that much harder to reliably replicate the VMS's entropy values within the restrictions set by the word type length distribution.

The VMS places stringent limits on overall word type length—less than 13 glyphs for all but extremely rare words, as calculated after Stolfi (2000)—and its observed word grammar and entropy values indicate that glyphs are paired up more predictably than alphabet letters are in natural languages (Zandbergen 2024), suggesting that if the VMS is a ciphertext, the cipher is probably verbose (Fisk et al. 2024). There

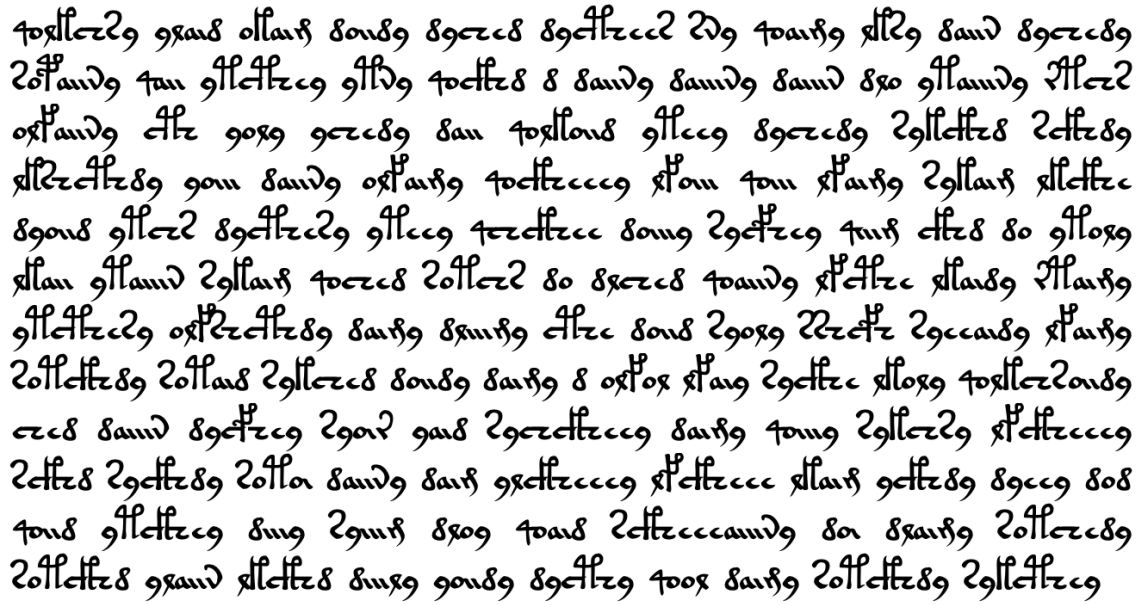
is a limit on how many natural-language plaintext letters one can cram into the average VMS token using a substitution cipher without causing the resulting ciphertext to have either higher average entropies or higher average word type lengths than what is observed in the VMS. Abbreviation ciphers are not a way out of this swamp, as the plaintext letters that remain after abbreviation are by their very nature of higher informational value than the word parts that were just abbreviated—a critical reason why the abbreviations could be done at all (Gheuens 2024). Relying heavily on abbreviations risks complicating the ability to achieve low conditional character entropy across an entire ciphertext token, and therefore across an entire ciphertext.

Even if an abbreviation cipher could be achieved that replicates various VMS properties when applied to a given plaintext, it would present a serious fine-tuning problem: How dependent would such a VMS-mimic cipher be on the diction of a particular plaintext? Could it deliver VMS-like ciphertext reliably across a range of plausible plaintexts within the same natural language? If not, then the immediate corollary is that if the VMS had been encrypted using an abbreviation cipher, then the probability of us seeing the VMS with the properties it has would have been low.

Overall, Voynichesque suggests that if the VMS is a ciphertext, each token probably represents a plaintext n-gram no more than ~6 letters long, with the vast majority being 1–2 letters long. This result reproduces one of the tacit results of Hermes (2022): that an artificial-language cipher that acts on a plaintext with many short n-grams (or, in the case of Polygraphia III, entirely unigrams) can deliver broadly VMS-like ciphertext behavior. However, with Voynichesque, this result emerges in a context with a far closer resemblance to the VMS—and is validated to an extraordinary degree by the performance of the Naibbe cipher.



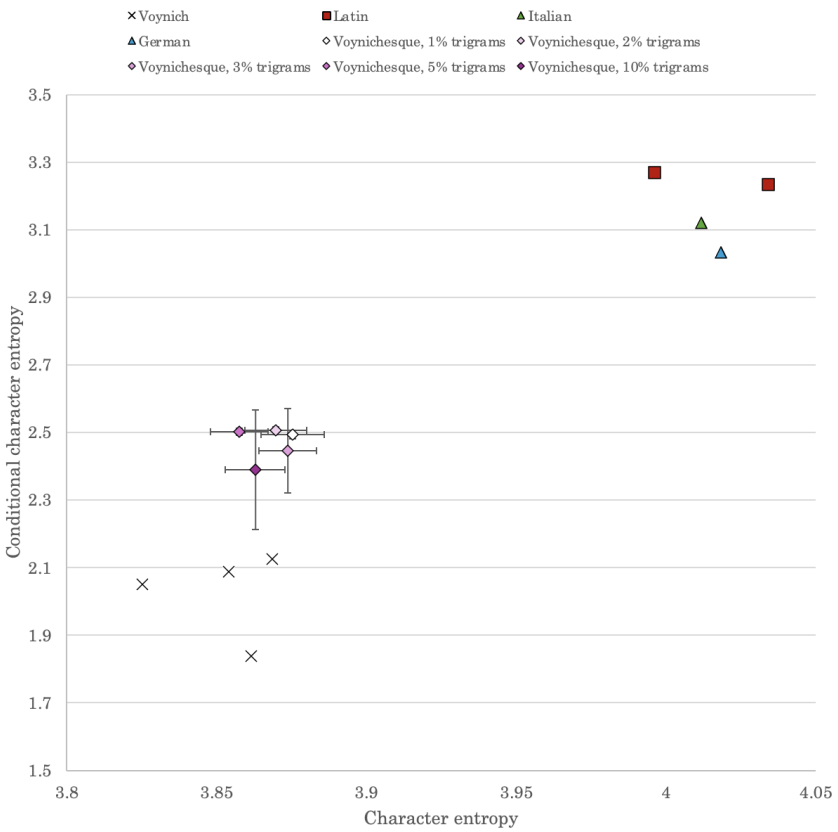
S2 Figure 1. A toy-model cipher can approximate the VMS's token length distribution if: the ciphertext is broken entirely into bigrams; each plaintext letter can be encrypted as one of four options, each of lengths 1–4 glyphs long; and the options that are 2 or 3 glyphs long are chosen 2–3 times more often than the options that are 1 or 4 glyphs long. Here, our toy-model cipher's letter substitution options are chosen in 1:2:2:1 ratios, where glyph lengths 2 or 3 are chosen twice as often as glyph lengths 1 or 4.



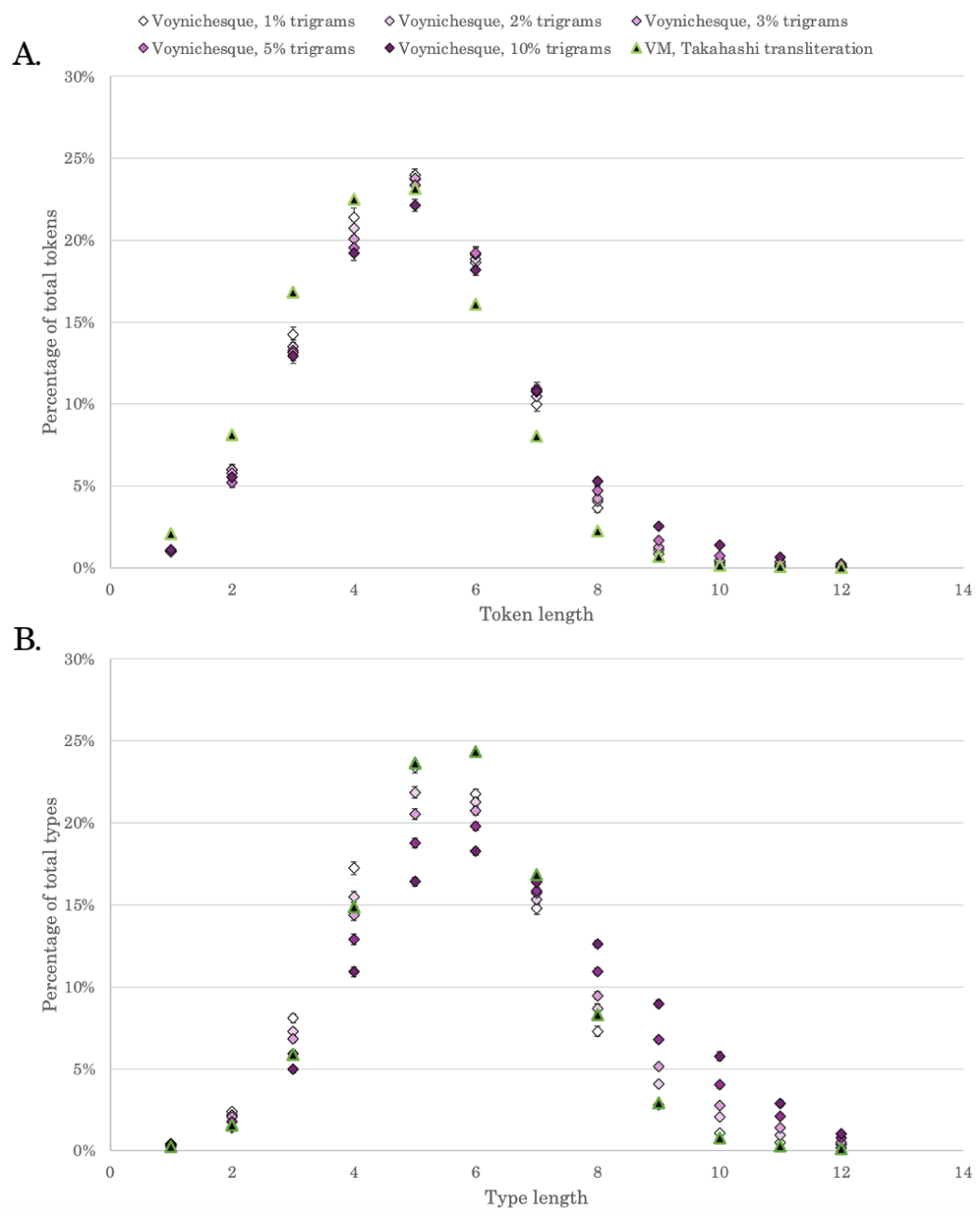
S2 Figure 2. A representative Voynichesque ciphertext. This ciphertext is the result of applying three randomly generated cipher alphabets based on the Zattera (2022) slot grammar onto the first 34 words of Book 16 of Pliny's *Natural History*, stripped of their original spacing and punctuation and respaced into unigrams, bigrams, and trigrams. See S2 Table 1 for this Voynichesque variant's cipher alphabets. The original plaintext here is: *Pomiferae arbores quaeque mitioribus sucis voluptatem primae cibis attulerunt et necessario alimento delicias miscere docuerunt, sive illae ultro ab homine didicere blandos sapores adoptione et conubio — idque munus etiam feris volucribusque dedimus — intra praedict[us]…*

Plaintext letter	Alphabet 1	Alphabet 2	Alphabet 3	Alphabet 3, unigram
A	*<dy>*	*<shcthd>*	* *	*<dol>*
B	*<s>*	*<d>*	*<iiid>*	*<diiid>*
C	*<syk>*	*<cphed>*	*<o>*	*<do>*
D	*<sot>*	*<cfhe>*	*<ai>*	*<dai>*
E	*<q>*	*<ched>*	*<aim>*	*<daim>*
F	*<ok>*	*<cheeed>*	*<od>*	*<dod>*
G	*<dyr>*	*<chcths>*	*<oiir>*	*<doiir>*
H	*<sl>*	*<shcfh>*	*<i>*	*<di>*
I	*<yt>*	*<ckhd>*	*<aid>*	*<daid>*
L	*<olf>*	*<cthes>*	*<iim>*	*<diim>*
M	*<yl>*	*<ee>*	*<aii>*	*<daii>*
N	*<lf>*	*<chckhee>*	*<iiil>*	*<diiil>*
O	*<sy>*	*<chs>*	*<n>*	*<dn>*
P	*<qolk>*	*<cth>*	*<oir>*	*<doir>*
Q	*<sop>*	*<s>*	*<oi>*	*<doi>*
R	*<qo>*	*<cthees>*	*<oiid>*	*<doiid>*
S	*<lk>*	*<ckhe>*	*<aiiin>*	*<daiiin>*
T	*<y>*	*<cthe>*	*<oiii>*	*<doiii>*
U	*<dl>*	*<ckheee>*	*<aiin>*	*<daiin>*
V	*<rt>*	*<cphee>*	*<iii>*	*<diii>*
X	*<dot>*	*<ctheees>*	*<iil>*	*<diil>*
Y	*<k>*	*<eeed>*	*<oin>*	*<doin>*
Z	*<yf>*	*<cphe>*	*<ir>*	*<dir>*

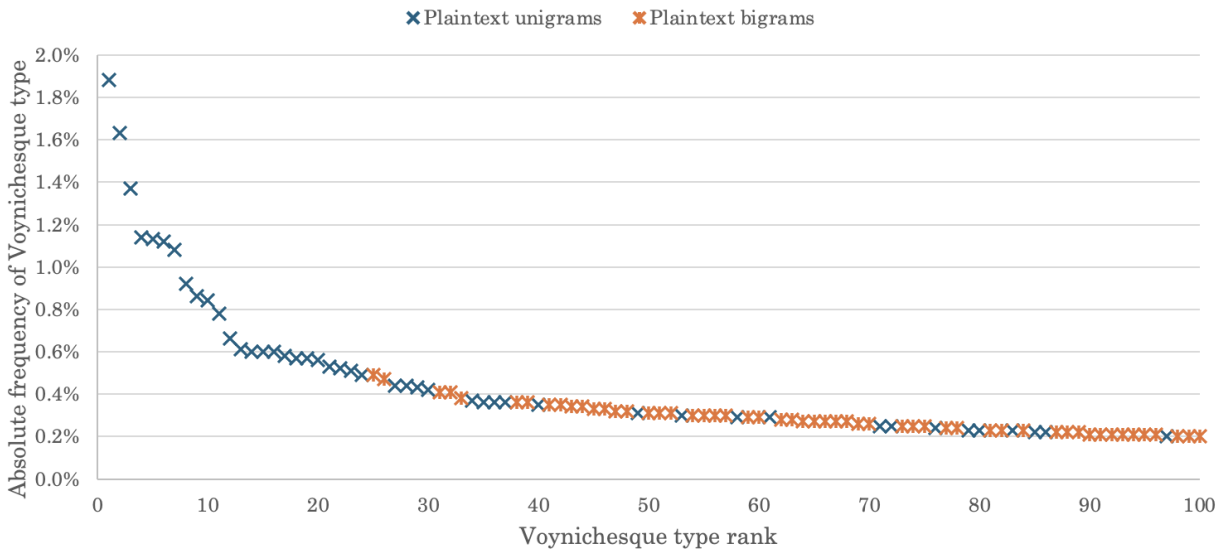
S2 Table 1. The randomly generated Voynichesque cipher alphabets used to create the ciphertext in S2 Figure 2. Note that the terminal *<y>* seen on many tokens in S2 Figure 2 is a null, applied on a token-by-token basis at a globally set probability.



S2 Figure 3. Randomly generated Voynichesque variants can achieve character entropies and conditional character entropies similar to those in the VM, as seen here in this reproduction of Zandbergen (2024b). Each Voynichesque data point represents the average output of 100 randomly generated ciphers all encrypting the same plaintext (Book 16 of Pliny's *Natural History*) for 10,000 tokens, with error bars representing a 95% confidence interval. All tuning parameters are identical across all Voynichesque runs represented here except those that govern respaced plaintext *n*-gram distribution. For a given trigram frequency *t*, the unigram frequency *u* was set as $u = \frac{1-t}{3}$ and the bigram frequency *b* was set as $b = \frac{2(1-t)}{3}$, such that all three frequencies always added up to 1. On average across all 100 runs, a given plaintext letter has a 15% chance of being replaced with a single Voynichese glyph, a 45% chance of being replaced with a two-glyph string, a 35% chance of being replaced with a three-glyph string, and a 5% chance of being replaced with a four-glyph string.



S2 Figure 4. Voynichesque variants can achieve token (a) and type (b) length distributions similar to the VMS's, but as the plaintext trigram frequency increases, these distributions rapidly become squatter and right-shifted into severe tension with the VMS. Each Voynichesque data point represents the average of 100 randomly generated ciphertexts, as described in the caption of S2 Figure 3. Error bars represent 95% confidence intervals.



S2 Figure 5. As seen in this plot based on a representative 10,000-token Voynichesque ciphertext, Voynichesque variants that encrypt unigram-bigram plaintexts create ciphertexts whose word types automatically show a Zipfian power-law relationship between a word type's frequency in the text and the word type's rank. This observation directly led to the modeling described in Supplementary Material 3 and the development of the Naibbe cipher.

Supplementary Material 3: Fitting the frequency-rank word type distribution of Voynich B with a model of a substitution cipher acting on plaintext unigrams

Voynichesque suggests that if the VMS is encrypted with a substitution cipher, then one favorable way to replicate the VMS's properties is to break up the plaintext into mostly unigrams and bigrams and then encrypt those unigrams and bigrams as ciphertext word types on a letter-by-letter basis. If the VMS is encrypted using this general kind of substitution cipher, Voynichesque also implies that the VMS's commonest word types may all represent plaintext unigrams. If so, these words' frequency-rank distribution would be expected to result from the average number of substitution options that can be applied to a given plaintext alphabet letter, the average ratios in which the options are applied, and the plaintext alphabet letter frequencies. I now describe a procedure by which this scenario can be modeled and fitted to Voynich B's observed frequency-rank distribution of its commonest word types.

S3.1. Model definition and design

My model takes the kth commonest plaintext letter with a plaintext letter frequency f_k and simulates the application of 6 different substitution options onto the letter. We do so by defining a linear combination where f_k equals the sum of six terms that each multiply f_k by a constant that represents the probability of applying a given substitution option:

$$f_k = \left(\frac{r_1}{R}\right) f_k + \left(\frac{r_2}{R}\right) f_k + \left(\frac{r_3}{R}\right) f_k + \left(\frac{r_4}{R}\right) f_k + \left(\frac{r_5}{R}\right) f_k + \left(\frac{r_6}{R}\right) f_k \qquad (1)$$

where $R = r_1 + r_2 + r_3 + r_4 + r_5 + r_6$.

We also define the ciphertext word frequency:

$$\omega_{j,k} = \left(\frac{r_j}{R}\right) f_k \qquad (2)$$

Where j and k are natural numbers and $1 \leq j \leq 6$ and $1 \leq k \leq 23$. That is, $\omega_{j,k}$ is the frequency of the ciphertext word type created by applying the jth substitution option onto the kth commonest alphabet letter.

We will be simulating 23 letters, to provide sensitivity to alphabets based on the 23-letter Latin alphabet. We choose 6 substitution options here, after exploratory tests of the model suggested that we needed only 6 options to fit the Voynich B distribution within our observing window (the 70 commonest word types; see below). But in principle, this setup could be changed to arbitrary numbers of options. Let us also define ω^* as the largest value of $\omega_{j,k}$ for all values of j and k and $\varpi_{j,k}$ as:

$$\varpi_{j,k} = \frac{\omega_{j,k}}{\omega^*} \tag{3}$$

We then rank all values $\varpi_{j,k}$ to create a proportional frequency-rank distribution of our model's simulated ciphertext word types. If two word types have the same initially calculated rank, we arbitrarily give the higher rank to the word type with the lower cipher number j, such that every ciphertext word type $\omega_{j,k}$ has a unique rank.

We require that r_1, r_2, r_3, r_4, r_5, and r_6—henceforth the option ratios—all be nonnegative, but we do not require them to be nonzero. This provides us with the flexibility to make use of fewer than 6 substitution options, and at the first sign that an option is not necessary to achieve goodness of fit, we will set that option ratio to 0. Likewise, we do not require that our letter frequencies add up to 1 (providing sensitivity to alphabets with more than 23 letters), nor do we require that letter frequencies be nonzero (providing sensitivity to alphabets with fewer than 23 letters). If letter frequencies add up to more than 1, we renormalize. Next, using GRG nonlinear optimization within Microsoft Excel, we perform a sum-of-least-squares best fit of our model to Voynich B's frequency-rank distribution for its top 70 word types as follows:

1. Initialize the model with letter frequencies and option ratios.
2. Rank the word types by their initial proportional frequencies and break rank ties.
3. Fit the option ratios while holding the letter frequencies constant.
4. Rerank the word types and break rank ties.
5. Fit the letter frequencies while holding the option ratios constant.
6. Rerank the word types and break rank ties.
7. Repeat steps 3-6 until a global best fit is achieved.

We perform the best fit on the 70 commonest word types because as demonstrated in Lindemann (2022), the proportional share of the 70 commonest words of all observed tokens shows maximum variance between natural languages, so testing the model fit in this window may provide an interesting viability test of the idea that all 70 of the commonest Voynich B words can be plaintext unigrams in disguise.

S3.2. Results of fitting the cipher model to Voynich B's 70 commonest word types

When my model is fitted to Voynich B's proportional frequency-rank distribution of word types, the letter frequencies that provide the best fit more closely resemble those of Romance languages than of some non-Romance language families, such as Slavic languages. The letter frequencies of Latin and late medieval Italian, as calculated from Dante's *Divina Commedia*, provide excellent fits.

S3.2.1. Initializing the model with a multi-language average

We begin by initializing our model with the average absolute letter frequencies of the 23 commonest letters of English, French, German, Czech, Spanish, Finnish, Portuguese, Italian, Danish, Polish, Swedish, Dutch, and Icelandic after stripping those letters of

diacritics (Wikipedia 2025; S3 Tables 1 and 2). To initialize the model with inequality in option application, we initialize the option ratios as $r_1 = 6$, $r_2 = 5$, $r_3 = 4$, $r_4 = 3$, $r_5 = 2$, and $r_6 = 1$. The model's resulting best-fit letter frequencies tend to cluster with those of Romance languages, with three of the top four closest matches being Portuguese, Spanish, and Italian (S3 Tables 3–5).

S3.2.2. Initializing with Latin

Our first attempt to fit our model was based on letter frequencies averaged out across multiple divergent European language groups. This is not reflective of our stylistic and historical knowledge of the VMS, which places the VMS firmly within the tradition of late medieval herbals found within northern Italy and Germany (Zandbergen 2024c), many of which were written in Latin (D'Imperio 1978). When re-initialized with Latin letter frequencies, our model converges on an excellent fit of six options per letter applied in approximately 5:2:2:2:1:1 ratios, and the best-fit letter frequencies strongly resemble those of modern Italian.

I then calculated letter frequencies for the main text (i.e., removing canto titles) of Dante's *Divina Commedia* (Supplementary Table 6). When initialized with Latin and Italian, model-estimated frequencies that best fit Voynich B closely match those in the *Divina Commedia*, which outperforms all tested modern languages across all model runs (S3 Tables 3–5).

S3.2.3. Initializing with German

When initialized with the German alphabet, the fitting procedure alters the letter frequencies to more closely resemble Spanish and Portuguese, thereby reinforcing that Latin and Romance languages provide better fits (Supplementary Table 2c). To generate

the best possible match to Voynich B, the three commonest letters in the model's plaintext alphabet need to occur in roughly 12:11:10 ratios, and the commonest letter needs to have a frequency of 11–12%, which is observed in Latin and Italian and not in German (S3 Tables 4 and 5).

S3.3. Universal recovery of Voynich B's frequency-rank word type distribution

In every case where we fit our hypothetical cipher model against the proportional frequency-rank word type distribution of Voynich B, we can recover the absolute frequency-rank word type distribution of Voynich B by dividing our best-fit ciphertext word frequencies almost exactly in half. In other words, we universally recover Voynich B's absolute word type distribution from our model of the proportional distribution if we assume that Voynich B is a ciphertext whose plaintext is ~50% unigrams. Our work with Voynichesque, as described in Supplementary Material 2, suggests that nearly all the remaining ~50% of the hypothetical Voynich B plaintext probably would consist of bigrams.

The combination of the six substitution options, the approximate 5:2:2:2:1:1 ratios, the excellent fit to Latin and Italian alphabet letter frequencies, and the model's implied 50-50 split between plaintext unigrams and plaintext bigrams directly led to the development of the Naibbe cipher as described in Section 2 of the main text. The Naibbe cipher's performance at mimicking the VMS when encrypting Latin and Italian, as outlined in the main text's Section 3, serves as a striking validation of this fit. This modeling effort also represents an unusual test of the VMS's compatibility with being a ciphertext—one that Voynich B, remarkably, passes. There is no intrinsic reason why the model's best-fit alphabet letter frequencies must resemble those of natural languages, much less those of Latin and Italian specifically.

Rank of letter frequency, no diacritics	English	French	German	Czech	Spanish	Finnish	Portuguese	Italian
1	12.70%	16.72%	16.40%	9.42%	14.14%	15.80%	16.12%	12.38%
2	9.06%	8.17%	9.78%	9.29%	12.03%	10.82%	13.89%	12.06%
3	8.17%	7.95%	7.27%	7.72%	9.51%	8.83%	10.71%	10.20%
4	7.51%	7.58%	7.09%	6.72%	7.98%	8.75%	6.81%	9.83%
5	6.97%	7.24%	7.00%	6.48%	7.02%	7.97%	6.53%	6.88%
6	6.75%	7.10%	6.55%	5.90%	6.97%	7.86%	6.32%	6.51%
7	6.33%	6.69%	6.15%	5.73%	6.87%	6.06%	4.99%	6.37%
8	6.09%	6.43%	5.16%	5.34%	5.01%	5.76%	4.74%	5.62%
9	5.99%	5.82%	5.08%	5.18%	4.97%	5.01%	4.45%	4.98%
10	4.25%	5.46%	4.58%	3.80%	4.63%	4.97%	4.41%	4.50%
11	4.03%	3.67%	3.44%	3.49%	4.11%	3.20%	4.34%	3.74%
12	2.78%	3.35%	3.04%	2.89%	4.02%	2.87%	3.87%	3.15%
13	2.76%	2.97%	3.01%	2.45%	3.16%	2.25%	2.78%	3.06%
14	2.41%	2.52%	2.73%	2.41%	2.51%	2.04%	2.52%	2.51%
15	2.36%	1.84%	2.53%	2.32%	2.22%	1.85%	1.58%	2.10%
16	2.23%	1.36%	1.92%	2.04%	1.97%	1.84%	1.30%	1.64%
17	2.02%	1.07%	1.89%	1.91%	1.77%	1.75%	1.28%	1.18%
18	1.97%	0.94%	1.66%	1.43%	1.43%	1.04%	1.20%	1.15%
19	1.93%	0.90%	1.42%	1.36%	1.14%	0.39%	1.04%	0.93%
20	1.49%	0.87%	1.13%	1.20%	0.88%	0.28%	1.02%	0.51%
21	0.98%	0.81%	0.85%	0.82%	0.69%	0.28%	0.47%	0.14%
22	0.77%	0.71%	0.67%	0.09%	0.52%	0.19%	0.45%	0.03%
23	0.15%	0.43%	0.31%	0.08%	0.49%	0.09%	0.38%	0.02%

S3 Table 1. Ranks of the 23 commonest alphabet letters, after removing diacritics, for some of the languages used for modeling Voynich B's frequency-rank type distribution. These letter frequencies are computed from Wikipedia (2025).

Rank of letter frequency, no diacritics	Danish	Polish	Swedish	Dutch	Icelandic	Average	Latin (Natural History, Book 16)	Late Medieval Italian (Divina Commedia)
1	15.45%	9.99%	12.52%	18.91%	11.91%	14.03%	11.47%	12.06%
2	8.96%	9.05%	10.15%	10.03%	9.15%	10.19%	11.01%	10.74%
3	7.24%	8.41%	8.54%	7.49%	8.58%	10.19%	9.91%	10.27%
4	7.22%	8.29%	8.43%	6.79%	7.71%	8.51%	8.11%	9.58%
5	6.86%	6.57%	7.69%	6.50%	7.07%	6.98%	7.56%	6.58%
6	6.00%	5.79%	6.59%	6.41%	5.63%	6.49%	7.05%	6.44%
7	5.86%	4.95%	5.82%	6.06%	5.18%	5.93%	7.04%	5.83%
8	5.81%	4.57%	5.79%	5.93%	4.95%	5.48%	6.51%	5.65%
9	5.58%	4.55%	5.28%	3.73%	4.53%	5.01%	5.27%	5.62%
10	5.23%	4.44%	4.70%	3.57%	4.39%	4.53%	5.00%	5.17%
11	4.08%	3.97%	3.47%	3.40%	4.24%	3.78%	4.32%	3.73%
12	3.40%	3.88%	3.14%	2.85%	4.04%	3.33%	3.45%	3.62%
13	3.24%	3.86%	2.86%	2.38%	3.94%	2.98%	2.67%	3.00%
14	2.41%	3.41%	2.42%	2.25%	3.31%	2.57%	2.49%	2.80%
15	2.33%	3.29%	2.09%	2.21%	3.01%	2.29%	1.63%	2.03%
16	2.00%	3.10%	2.03%	1.99%	2.44%	1.99%	1.50%	1.82%
17	1.98%	2.91%	1.92%	1.58%	1.87%	1.78%	1.21%	1.77%
18	1.76%	2.35%	1.84%	1.57%	1.58%	1.53%	1.19%	1.27%
19	1.62%	2.34%	1.54%	1.52%	1.46%	1.35%	1.17%	0.83%
20	0.87%	1.48%	1.49%	1.46%	1.14%	1.06%	0.65%	0.74%
21	0.73%	1.38%	0.71%	1.39%	1.13%	0.80%	0.65%	0.46%
22	0.70%	1.07%	0.61%	1.24%	1.04%	0.62%	0.12%	0.00%
23	0.57%	0.31%	0.16%	0.81%	0.87%	0.36%	0.01%	0.00%

S3 Table 2. Ranks of the 23 commonest alphabet letters, after removing diacritics, for some of the languages used for modeling Voynich B's frequency-rank type distribution. Many of these languages' letter frequencies are computed from Wikipedia (2025), with Latin computed from Book 16 of Pliny's *Natural History* (Thayer 2025) and late medieval Italian computed from Dante's *Divina Commedia* (Project Gutenberg 1997).

Ranked match versus Voynich B modeled letter frequencies	Language	Language group	Sum of least squares versus model output
1	Portuguese	Romance	0.001355
2	Spanish	Romance	0.002564
3	Finnish	Nordic	0.002935
4	Italian	Romance	0.003519
5	German	West Germanic	0.003994
6	Dutch	West Germanic	0.004380
7	Swedish	Nordic	0.004506
8	Danish	Nordic	0.004958
9	French	Romance	0.005950
10	Icelandic	Nordic	0.006070
11	English	West Germanic	0.006316
12	Polish	West Slavic	0.008946
13	Czech	West Slavic	0.009796

S3 Table 3. Comparisons between the cipher model's best-fit alphabet letter frequencies after fitting Voynich B's frequency-rank type distribution, and the alphabet letter frequencies (no diacritics) of various European languages, when the model was initialized with a multi-language average.

Ranked match versus Voynich B modeled letter frequencies	Language	Language group	Sum of least squares versus model output
1	Late medieval Italian	Romance	0.000743
2	Swedish	Nordic	0.000929
3	Italian	Romance	0.001175
4	English	West Germanic	0.001653
5	Spanish	Romance	0.001747
6	Finnish	Nordic	0.001920
7	Icelandic	Nordic	0.002823
8	Czech	West Slavic	0.003056
9	Danish	Nordic	0.003098
10	French	Romance	0.003381
11	German	West Germanic	0.003723
12	Polish	West Slavic	0.004226
13	Portuguese	Romance	0.004709
14	Dutch	West Germanic	0.007481

S3 Table 4. Comparisons between the cipher model's best-fit alphabet letter frequencies after fitting Voynich B's frequency-rank type distribution, and the alphabet letter frequencies (no diacritics) of various European languages, when the model was initialized with Latin.

Ranked match versus Voynich B modeled letter frequencies	Language	Language group	Sum of least squares versus model output
1	Portuguese	Romance	0.001296
2	Spanish	Romance	0.002252
3	Finnish	Nordic	0.002659
4	German	West Germanic	0.003177
5	Italian	Romance	0.003197
6	Swedish	Nordic	0.003677
7	Dutch	West Germanic	0.003702
8	Late medieval Italian	Romance	0.003924
9	Danish	Nordic	0.004002
10	Icelandic	Nordic	0.004890
11	English	West Germanic	0.005241
12	French	West Germanic	0.005269
13	Polish	West Slavic	0.007511
14	Czech	West Slavic	0.008091

S3 Table 5. Comparisons between the cipher model's best-fit alphabet letter frequencies after fitting Voynich B's frequency-rank type distribution, and the alphabet letter frequencies (no diacritics) of various European languages, when the model was initialized with German.

Supplementary Material 4: Random walk analysis

See Peng et al. (1992), Kokol et al. (1999), and Schinner (2007) for a more thorough treatment of the generals of random walk analysis and its specific applications to text, but in essence, this analysis assigns each alphabet letter within a text a unique, 5-digit code made of 0s and 1s (the exact codes do not matter that much), and then each 1 or 0 is treated as a "step" forward (1) or backward (-1), respectively, on a "walk." If this walk were truly random over the course of the entire binary sequence, then the direction of any one step would be independent of previous steps. However, if long-range correlations exist within a given binary sequence, the direction of a given step depends on the directions of previous steps, amounting to a "memory" effect.

Taking after Peng et al. (1992), let us define the walker's *i*th step as resulting in the unit step $u(i)$, where $u(i) = \pm 1$. We also define $y(l)$ as being the walker's net displacement after l steps:

$$y(l) = \sum_{i=1}^{l} u(i)$$

For example, for the binary sequence {01101}, $y(4) = 0$, and $y(5) = 1$.

Let us also define l_0 as a binary sequence position that marks the start of l steps. We define $\Delta y(l)$ as:

$$\Delta y(l) = y(l_0 + l) - y(l_0)$$

Returning to our example binary sequence {01101}, for $l_0 = 3$, $\Delta y(1)$ is given as:

$$\Delta y(1) = y(4) - y(3) = 0 - 1 = -1$$

We also define the mean square fluctuation $F^2(l)$ as:

$$F^2(l) = \overline{[\Delta y(l)]^2} - \left[\overline{\Delta y(l)}\right]^2$$

Where a horizontal bar denotes average values across all positions l_0. The root mean square fluctuation $F(l)$ is simply $F(l) = \sqrt{F^2(l)}$.

As Peng et al. (1992) helpfully put it, what we are basically doing is fixing a set of calipers of width l and then going down the binary sequence and treating each bit as l_0. Each time we stop at a given l_0, we calculate $\Delta y(l)$ and its square. In plain English, $F^2(l)$ is the difference between the average of the squares of all values of $\Delta y(l)$ and the square of the average of all values of $\Delta y(l)$. For a fully random walk, the root mean square fluctuation F(l) in the displacement of a walk of l number of steps, each of length ±1, would be expected to approximate $\sqrt{l}$, especially at large values of l. However, if $F(l) > \sqrt{l}$, then long-range correlations of some kind, and to some degree, are at work.

References

Burnett, C. 2020. “The Palaeography of Numerals.” In *The Oxford Handbook of Latin Palaeography*, edited by Frank T. Coulson and Robert G. Babcock, 25–36. Oxford: Oxford University Press.

Caruana, A., C. Layfield, , and J. Abela. 2022. “An Analysis of the Relationship between Words within the Voynich Manuscript.” Conference paper, International Conference on the Voynich Manuscript 2022, November 30–December 1, 2022, University of Malta. https://ceur-ws.org/Vol-3313/paper8.pdf

Davis, L.F. 2020. “How Many Glyphs and How Many Scribes? Digital Paleography and the Voynich Manuscript.” *Manuscript Studies* 5 (1): 164-180. https://doi.org/10.1353/mns.2020.0011.

Enderle, S. 2017. “Topic Modeling Tool.” https://github.com/senderle/topic-modeling-tool.

Fisk, D, K. Gheuens, R. Zandbergen, et al. 2024. “Voynich text generation.” *Voynich Ninja*. 2019–2024. Accessed 15 May 2025. https://www.voynich.ninja/thread-2684-page-7.html?highlight=verbose+homophonic.

Gheuens, K. 2024. “Why Your Voynich Theory is Wrong (pt. 1): ‘It’s the Entropy, Stupid!’” *Voynich Talk*. 10 September 2024. Accessed 10 May 2025. https://www.youtube.com/watch?v=XSTM8Gixai4.

Hermes, J. “Polygraphia III: The cipher that pretends to be an artificial language.” Conference paper, International Conference on the Voynich Manuscript 2022, November 30–December 1, 2022, University of Malta. https://ceur-ws.org/Vol-3313/paper7.pdf.

Holmes, T. Rice ed. “C. Julius Caesar, De bello Gallico.” Accessed 15 September 2025. https://www.perseus.tufts.edu/hopper/text?doc=Perseus:text:1999.02.0002

King, P. 2008. “Scholarly Resources.” Peter King’s Website. Accessed 10 May 2025. http://individual.utoronto.ca/pking/resources.html.

Reddy, S., and K. Knight. 2011. “What We Know About The Voynich Manuscript.” *Proceedings of the 5th ACL-HLT Workshop on Language Technology for Cultural Heritage, Social Sciences, and Humanities*: 78-86. https://aclanthology.org/W11-1511/.

Kokol, P., V. Podgorelec, M. Zorman, T. Kokol, and T. Njivar. 1999. “Computer and Natural Language Texts - A Comparison Based on Long-Range Correlations.” *Journal of the American Society for Information Science*. 50 (14): 1295-1301.

Levenshtein, V.I. 1966. “Binary codes capable of correcting deletions, insertions, and reversals.” *Soviet Physics Doklady* 10 (8):707–710.

McCallum, A.K. "MALLET: A Machine Learning for Language Toolkit." http://mallet.cs.umass.edu. 2002.

Peng, C.K., S. Buldyrev, A. Goldberger, et al. 1992. “Long-range correlations in nucleotide sequences.” *Nature* 356, 168–170. https://doi.org/10.1038/356168a0.

Ponzi, M. “A transcription and translation of the so-called ‘Alchemical herbal.’” *ViridisGreen*. 15 December 2023. Accessed 10 May 2025. https://medium.com/viridisgreen/a-transcription-and-translation-of-the-so-called-alchemical-herbal-dcb741200488.

Project Gutenberg. 1 August 1997. “The Project Gutenberg eBook of La Divina Commedia di Dante: Complete.” Accessed 10 May 2025. https://www.gutenberg.org/cache/epub/1000/pg1000-images.html.

Rousseeuw, P. J. 1987. “Silhouettes: A graphical aid to the interpretation and validation of cluster analysis.” *Journal of Computational and Applied Mathematics 20*: 53-65. https://doi.org/10.1016/0377-0427(87)90125-7.

Schinner, A. 2007. “The Voynich Manuscript: Evidence of the Hoax Hypothesis.” *Cryptologia* 31 (2): 95–107. https://doi.org/10.1080/01611190601133539.

Sterneck, R., A. Polish, and C.L. Bowern. 2021. “Topic Modeling in the Voynich Manuscript.” *arXiv*, 6 July 2021. https://doi.org/10.48550/arXiv.2107.02858.

Stolfi, Jorge. 2000. “On the VMS Word Length Distribution.” 23 December 2000. Accessed 9 May 2025. https://www.voynich.nu/hist/stolfi/word-length-distr.html.

Thayer, B. Pliny the Elder: Natural History, Book XVI. Accessed 10 May 2025. https://penelope.uchicago.edu/Thayer/L/Roman/Texts/Pliny_the_Elder/16*.html

Timm, T. 2015. “How the Voynich Manuscript was created.” *arXiv*, 9 December 2015. https://arxiv.org/abs/1407.6639.

Timm, T. 2016. “Co-Occurrence Patterns in the Voynich Manuscript.” *arXiv*, 8 February 2016. https://arxiv.org/abs/1601.07435.

Timm, T., and A. Schinner. 2020. “A possible generating algorithm of the Voynich manuscript.” *Cryptologia* 44 (1): 1–19. https://doi.org/10.1080/01611194.2019.1596999.

Timm, T., and A. Schinner. 2024. “The Voynich Manuscript: discussion of text creation hypotheses.” *Cryptologia* 48 (4): 305–322. https://doi.org/10.1080/01611194.2023.2225716.

Trithemius, J. *Polygraphia*. Book 3: 126–147. Cologne: apud Ioannem

Birckmannum & Vernerum Richwinum, 1564. https://www.google.com/books/edition/Polygraphia/ZiIaOe0OJnsC?hl=en&gbpv=1&printsec=frontcover.

Wikipedia. 2025. “Letter frequency.” Accessed 10 May 2025. https://en.wikipedia.org/wiki/Letter_frequency.

Zandbergen, R. 2024. “Analysis Section (4/5) - Word statistics.” *The Voynich Manuscript*. 18 December 2024. Accessed 10 May 2025.

Zattera, M. 2022. “A New Transliteration Alphabet Brings New Evidence of Word Structure and Multiple ‘Languages’ in the Voynich Manuscript” (Conference paper, International Conference on the Voynich Manuscript 2022, November 30–December 1, 2022, University of Malta). https://ceur-ws.org/Vol-3313/paper10.pdf.